OPERATING SYSTEMS AND SYSTEMS PROGRAMMING

OVERVIEW

By
Aidoo, Joshua Kweku

University of Mines and Technology (UMaT)

CY 375,
January 2026

Expected Leaning Outcomes

At the end of the course, Students should have gained an understanding in:

- Operating System Fundamentals and Architecture: Understand the core functions of operating systems, including layered architecture, system calls, interrupts, processes, and threads.
- Process Management and Synchronization: Explain concepts of multitasking, process scheduling, synchronization mechanisms, and protection issues in concurrent systems.
- Memory, I/O, and File System Management: Describe memory management techniques such as paging and virtual memory, along with input-output architecture, device management, and file systems.
- Modern Operating System Case Studies: Understand and compare the design principles of UNIX, Windows NT, and Linux operating systems.
- Distributed Operating System Concepts: Explain the fundamentals of distributed file systems, resource sharing, and distributed operating system environments.

Assessment Criteria

Main Assessment Criteria

- Class Attendance:**10%**
- Class Assessment:**30%**
  - Quizzes
  - Assignments (maybe)
  - Group Presentations
- End of Semester Examination:**60%**

Other Mark Earners

- Contribution in Class

Reference Material

At the end of the course, students should have gained an understanding in:

- Silberschatz, A., Galvin, P. B. and Gagne, G. (2018), Operating System Concepts, Wiley, 10th Edition, Hoboken, U.S.A., 976 pp. (primary reference)
- Dularey, E. (2010), Linux All-in-One for Dummies, Wiley Publishing Inc., 4th Edition, Hoboken, U.S.A., 652 pp.
- Nixon, R. (2010), Ubuntu: Up and Running, O'Reilly Media Inc., 1st Edition, Sebastopol, U.S.A., 464 pp.
- Lynn, S. (2012), Windows Server 2012: Up and Running, O'Reilly Media Inc., 1st Edition, Sebastopol, U.S.A., 258 pp.

Learning Objectives

General Organization of a Computer System and Interrupts

Components of Modern Multiprocessor Computer Systems

User Mode and Kernel Mode Transitions

Operating Systems in Various Computing Environments

Free and Open-Source Operating Systems

Introduction

Definition

An operating system is software that manages a computer's hardware. It also provides a basis for application programs and acts as an intermediary between the computer user and the computer hardware.

- Operating systems are everywhere, from cars and home appliances that include "Internet of Things" devices, to smart phones,personal computers, enterprise computers, and cloud computing environments.
- This includes the CPU, memory, and I/O devices, as well as storage.
- A fundamental responsibility of an operating system is to allocate these resources to programs.
- Because an operating system is large and complex, it must be created piece by piece.
- Each of these pieces should be a well-delineated portion of the system, with carefully defined inputs, outputs, and functions.

What Operating Systems Do

- A computer system can be divided roughly into four components: the hardware, the operating system, the application programs, and a user.
- The hardware—the central processing unit (CPU), the memory, and the input/output (I/O) devices—provides the basic computing resources for the system.
- The application programs—such as word processors, spreadsheets,compilers, and web browsers—define the ways in which these resources are used to solve users' computing problems.
- The operating system controls the hardware and coordinates its use among the various application programs for the various users.
- Like a government, it performs no useful function by itself. It simply provides an environment within which other programs can do useful work.
- To understand more fully the operating system's role, we next explore operating systems from two viewpoints: that of the **user** and that of the **system**.

What Operating Systems Do

User View

- The user's view of the computer varies according to the interface being used.
- Many computer users sit with a laptop or in front of a PC consisting of a monitor, keyboard, and mouse. Such a system is designed for one user to monopolize its resources.
- The goal is to maximize the work (or play) that the user is performing.
- In this case, the operating system is designed mostly for **ease of use**, with some attention paid to performance and security and none paid to **resource utilization**—how various hardware and software resources are shared.
- Increasingly, many users interact with mobile devices such as smartphones and tablets—devices that are replacing desktop and laptop computer systems for some users.

What Operating Systems Do

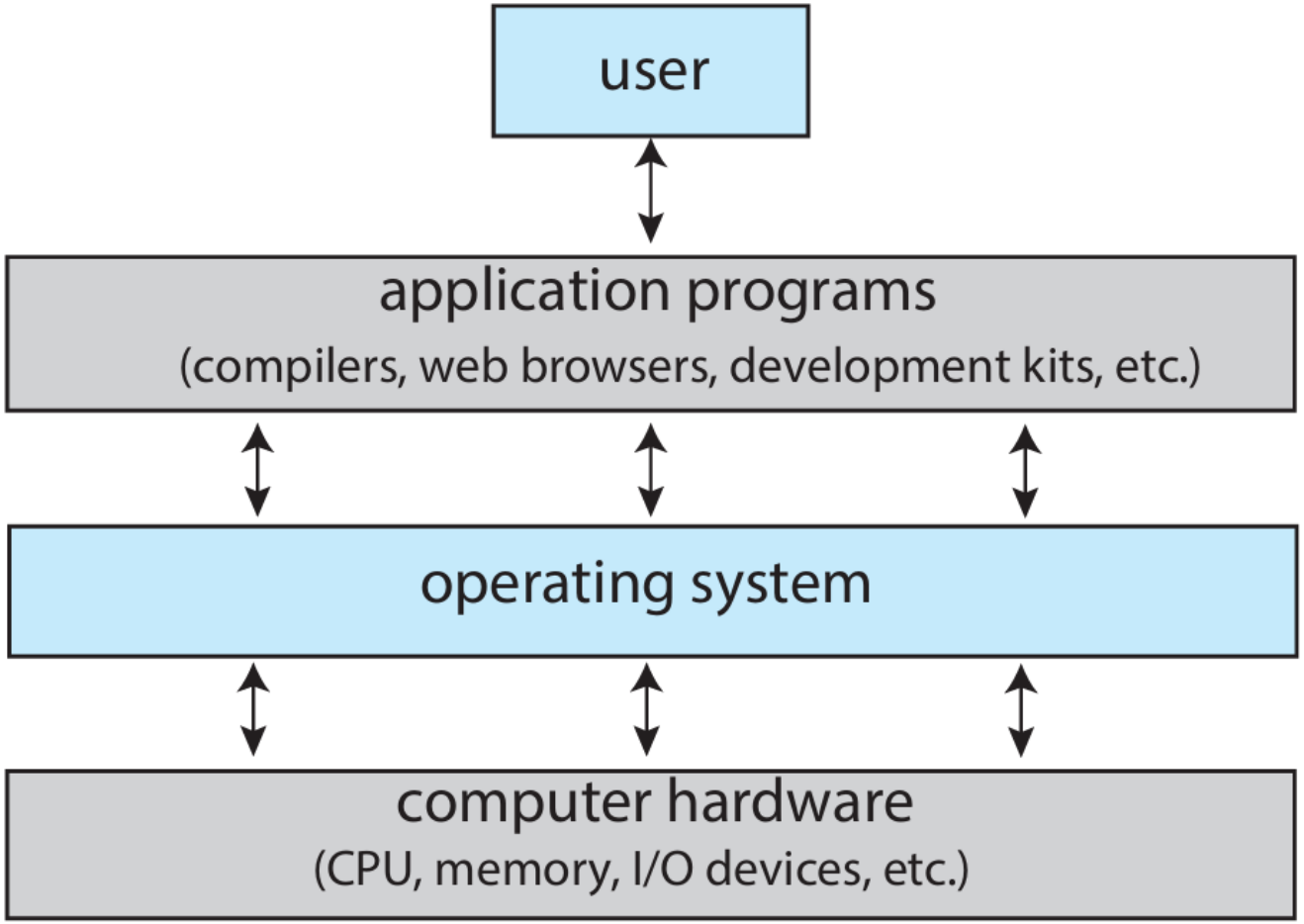


Figure 1: Abstract view of the components of a computer system

What Operating Systems Do

System View

- From the computer's point of view, the operating system is the program most intimately involved with the hardware.
- In this context, we can view an operating system as a resource allocator.
- A computer system has many resources that may be required to solve a problem: CPU time, memory space, storage space, I/O devices, and so on.
- The operating system acts as the manager of these resources.
- Facing numerous and possibly conflicting requests for resources, the operating system must decide how to allocate them to **specific programs and users** so that it can operate the computer system efficiently and fairly.
- The operating system is the one program running at all times on the computer—usually called the **kernel**.

Computer-System Organization

- A modern general-purpose computer system consists of one or more CPUs and a number of device controllers connected through a common bus.
- The bus provides access between components and shared memory.
- Each device controller is in charge of a specific type of device (for example, a disk drive, audio device, or graphics display).
- A device controller maintains some local buffer storage and a set of special-purpose registers. The device controller is responsible for moving the data between the peripheral devices that it controls and its local buffer storage.
- A device controller maintains some local buffer storage and a set of special-purpose registers.
- The device controller is responsible for moving the data between the peripheral devices that it controls and its local buffer storage.

Computer-System Organization

- Operating systems have a device driver for each device controller.
- This device driver understands the device controller and provides the rest of the operating system with a uniform interface to the device.

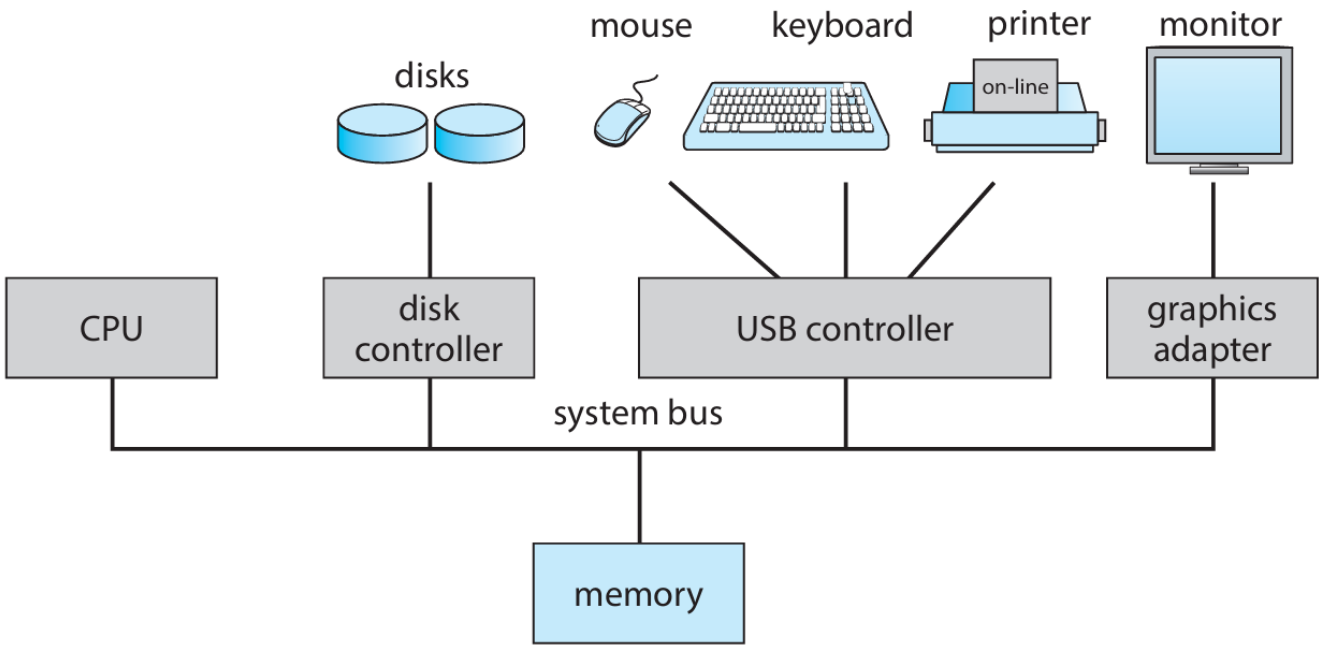
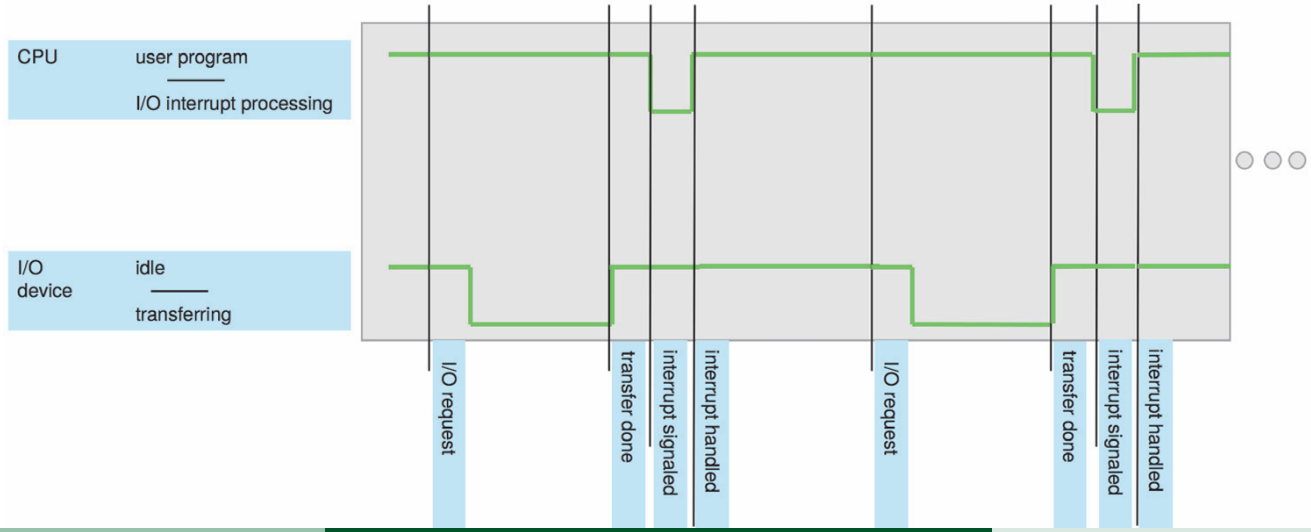


Figure 2: A typical PC computer system.

Interrupts

- Hardware may trigger an interrupt at any time by sending a signal to the CPU, usually by way of the system bus.
- Interrupts are used for many other purposes as well and are a key part of how operating systems and hardware interact.
- When the CPU is interrupted, it stops what it is doing and immediately transfers execution to a fixed location.
- The fixed location usually contains the starting address where the service routine for the interrupt is located.
- The interrupt service routine executes; on completion, the CPU resumes the interrupted computation.



Interrupts

- Interrupts must transfer control to the appropriate interrupt service routine.
- Tot mange the transfer would be to invoke a generic routine to examine the interrupt information. The routine, in turn, would call the interrupt-specific handler.
- However, interrupts must be handled quickly, as they occur very frequently. A table of pointers to interrupt routines can be used instead to provide the necessary speed.
- The interrupt routine is called indirectly through the table, with no intermediate routine needed. Generally, the table of pointers is stored in low memory (the first hundred or so locations).
- These locations hold the addresses of the interrupt service routines for the various devices.

Interrupts

- This array, or interrupt vector, of addresses is then indexed by a unique number, given with the interrupt request, to provide the address of the interrupt service routine for the interrupting device.
- Operating systems as different as Windows and UNIX dispatch interrupts in this manner.

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts

Figure 4: Intel processor event-vector table.

Interrupts

- The interrupt architecture must also save the state information of whatever was interrupted, so that it can restore this information after servicing the interrupt.
- If the interrupt routine needs to modify the processor state—for instance, by modifying register values—it must explicitly save the current state and then restore that state before returning.
- After the interrupt is serviced, the saved return address is loaded into the program counter, and the interrupted computation resumes as though the interrupt had not occurred.

Interrupts

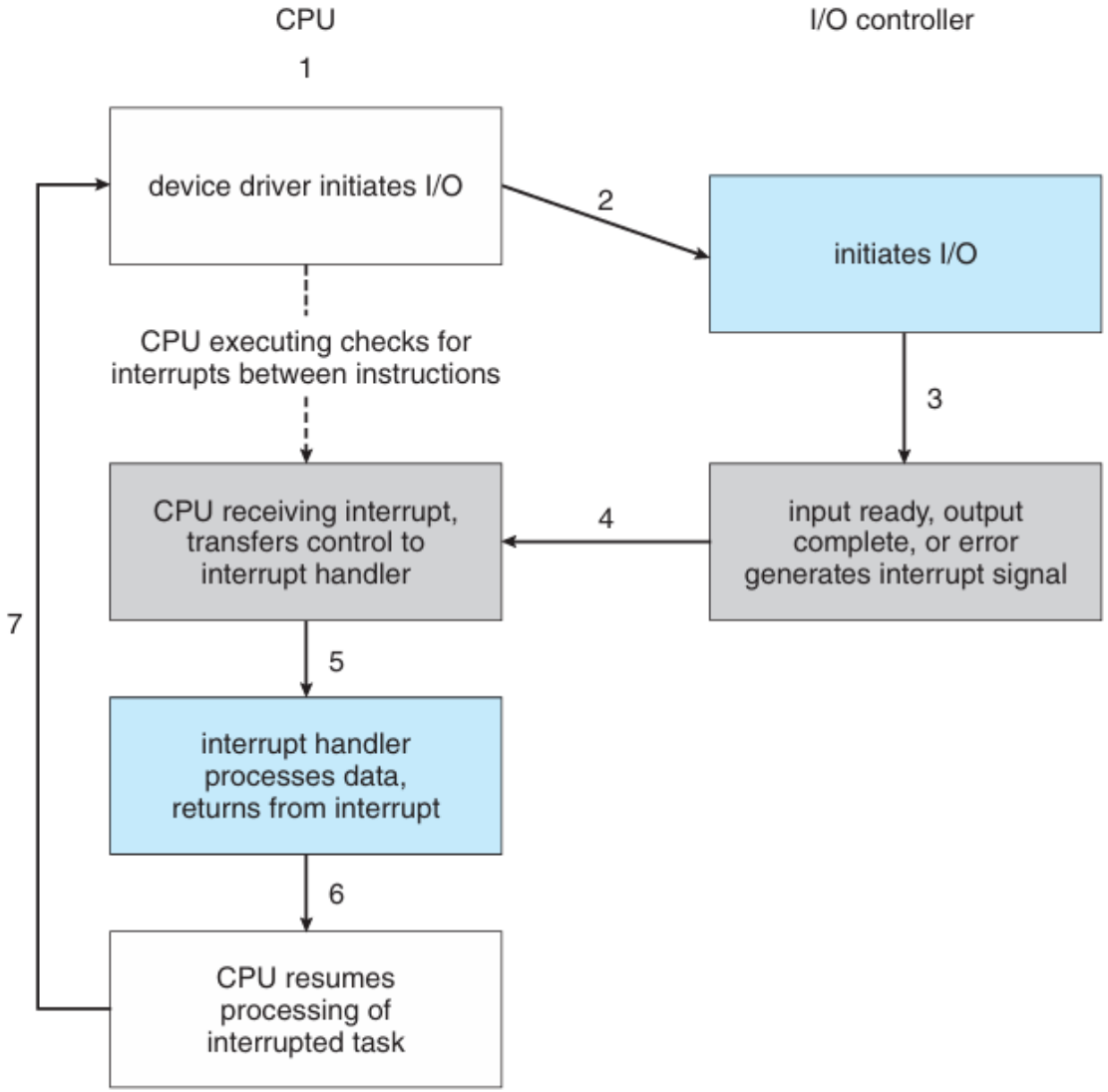


Figure 5: Interrupt-driven I/O cycle.

Interrupts

- Most CPUs have two interrupt request lines: nonmaskable interrupt and maskable interrupt.
- Nonmaskable interrupt, which is reserved for events such as unrecoverable memory errors.
- Maskable: it can be turned off by the CPU before the execution of critical instruction sequences that must not be interrupted. The maskable interrupt is used by device controllers to request service.
- The interrupt mechanism also implements a system of interrupt priority levels.
- These levels enable the CPU to defer the handling of low-priority interrupts without masking all interrupts and make it possible for a high-priority interrupt to preempt the execution of a low-priority interrupt.

Storage Structure

- The CPU can load instructions only from memory, so any programs must first be loaded into memory to run. General-purpose computers run most of their programs from rewritable memory, called main memory (also called random-access memory, or RAM). Main memory commonly is implemented in a semiconductor technology called dynamic random-access memory (DRAM).
- Computers use other forms of memory as well.
- For example, the first program to run on computer power-on is a bootstrap program, which then loads the operating system.
- The computer uses electrically erasable programmable read-only memory (EEPROM) and other forms of firmware—storage that is infrequently written to and is nonvolatile.

Storage Structure

- All forms of memory provide an array of bytes. Each byte has its own address.
- Interaction is achieved through a sequence of load or store instructions to specific memory addresses.
- The load instruction moves a byte or word from main memory to an internal register within the CPU, whereas the store instruction moves the content of a register to main memory.
- Aside from explicit loads and stores, the CPU automatically loads instructions from main memory for execution from the location stored in the program counter.
- Ideally, we want the programs and data to reside in main memory permanently. This arrangement usually is not possible on most systems for two reasons:
  - Main memory is usually too small to store all needed programs and data permanently.
  - Main memory, as mentioned, is volatile—it loses its contents when power is turned off or otherwise lost.

Storage Structure

- Thus, most computer systems provide secondary storage as an extension of main memory. The main requirement for secondary storage is that it be able to hold large quantities of data permanently.
- The most common secondary-storage devices are hard-disk drives (HDDs) and nonvolatile memory (NVM) devices, which provide storage for both programs and data.
- Most programs (system and application) are stored in secondary storage until they are loaded into memory.
- Many programs then use secondary storage as both the source and the destination of their processing.
- Secondary storage is also much slower than main memory.
- Other possible storage include cache memory, CD-ROM or blu-ray, magnetic tapes, and so on.
- Those that are slow enough and large enough that they are used only for special purposes—to store backup copies of material stored on other devices, for example—are called tertiary storage.

Storage Structure

- Thus, most computer systems provide secondary storage as an extension of main memory. The main requirement for secondary storage is that it be able to hold large quantities of data permanently.
- The most common secondary-storage devices are hard-disk drives (HDDs) and nonvolatile memory (NVM) devices, which provide storage for both programs and data.
- Most programs (system and application) are stored in secondary storage until they are loaded into memory.
- Many programs then use secondary storage as both the source and the destination of their processing.
- Secondary storage is also much slower than main memory.
- Other possible storage include cache memory, CD-ROM or blu-ray, magnetic tapes, and so on.
- Those that are slow enough and large enough that they are used only for special purposes—to store backup copies of material stored on other devices, for example—are called tertiary storage.

Storage Structure

- The top four levels of memory in the figure are constructed using semiconductor memory, which consists of semiconductor-based electronic circuits.
- NVM devices, at the fourth level, have several variants but in general are faster than hard disks.

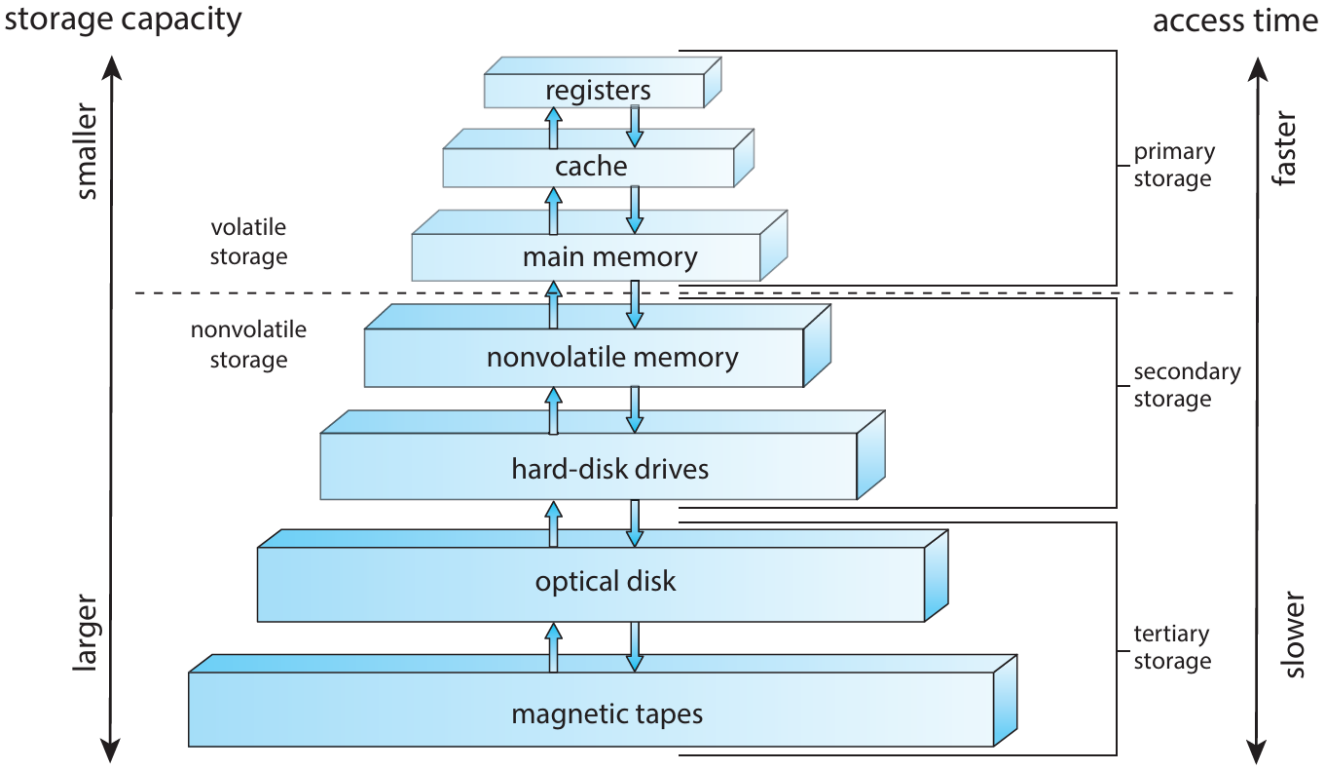


Figure 6: Storage-device hierarchy.

I/O Structure

- A large portion of operating system code is dedicated to managing I/O, both because of its importance to the reliability and performance of a system and because of the varying nature of the devices.
- General-purpose computer system consists of multiple devices, all of which exchange data via a common bus.
- The form of interrupt-driven I/O described earlier is fine for moving small amounts of data but can produce high overhead when used for bulk data movement such as NVS I/O.
- To solve this problem, direct memory access (DMA) is used.
- After setting up buffers, pointers, and counters for the I/O device, the device controller transfers an entire block of data directly to or from the device and main memory, with no intervention by the CPU.
- Only one interrupt is generated per block, to tell the device driver that the operation has completed, rather than the one interrupt per byte generated for low-speed devices.
- While the device controller is performing these operations, the CPU is available to accomplish other work.

I/O Structure

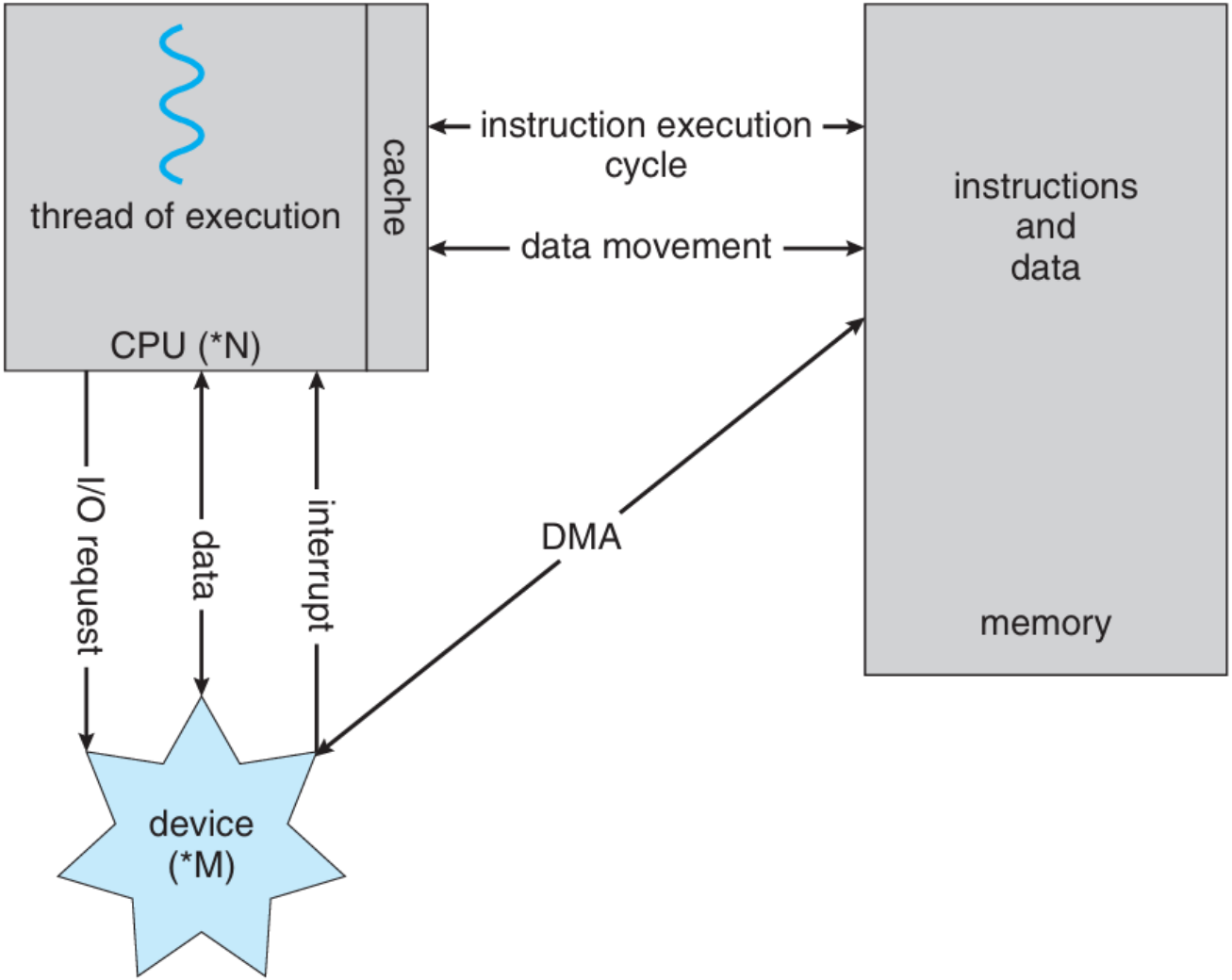


Figure 7: How a modern computer system works.

Single-Processor Systems

- Many years ago, most computer systems used a single processor containing one CPU with a single processing core.
- The core is the component that executes instructions and registers for storing data locally.
- The one main CPU with its core is capable of executing a general-purpose instruction set, including instructions from processes. These systems have other special-purpose processors as well.
- They may come in the form of device-specific processors, such as disk, keyboard, and graphics controllers.
- All of these special-purpose processors run a limited instruction set and do not run processes.
- Sometimes, they are managed by the operating system, in that the operating system sends them information about their next task and monitors their status.
- For example, a disk-controller microprocessor receives a sequence of requests from the main CPU core and implements its own disk queue and scheduling algorithm.

Single-Processor Systems

- This arrangement relieves the main CPU of the overhead of disk scheduling. PCs contain a microprocessor in the keyboard to convert the keystrokes into codes to be sent to the CPU.
- In other systems or circumstances, special-purpose processors are low-level components built into the hardware.
- The operating system cannot communicate with these processors; they do their jobs autonomously.
- The use of special-purpose microprocessors is common and does not turn a single-processor system into a multiprocessor.
- If there is only one general-purpose CPU with a single processing core, then the system is a single-processor system.

Multiprocessor Systems

- On modern computers, from mobile devices to servers, multiprocessor systems now dominate the landscape of computing.
- Traditionally, such systems have two (or more) processors, each with a single-core CPU.
- The processors share the computer bus and sometimes the clock, memory, and peripheral devices. The primary advantage of multiprocessor systems is increased throughput.
- That is, by increasing the number of processors, we expect to get more work done in less time.
- The speed-up ratio with N processors is not N, however; it is less than N.
- When multiple processors cooperate on a task, a certain amount of overhead is incurred in keeping all the parts working correctly.
- This overhead, plus contention for shared resources, lowers the expected gain from additional processors.

Multiprocessor Systems

- The most common multiprocessor systems use symmetric multiprocessing (SMP), in which each peer CPU processor performs all tasks, including operating-system functions and user processes.
- The definition of multiprocessor has evolved over time and now includes multicore systems, in which multiple computing cores reside on a single chip.
- Multicore systems can be more efficient than multiple chips with single cores because on-chip communication is faster than between-chip communication.

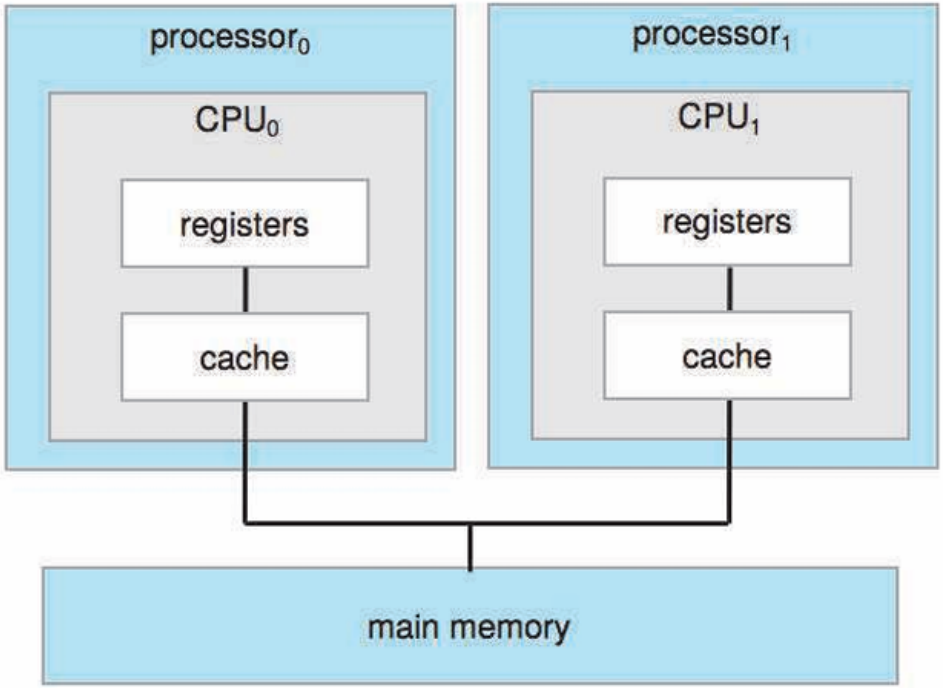


Figure 8: Symmetric multiprocessing architecture.

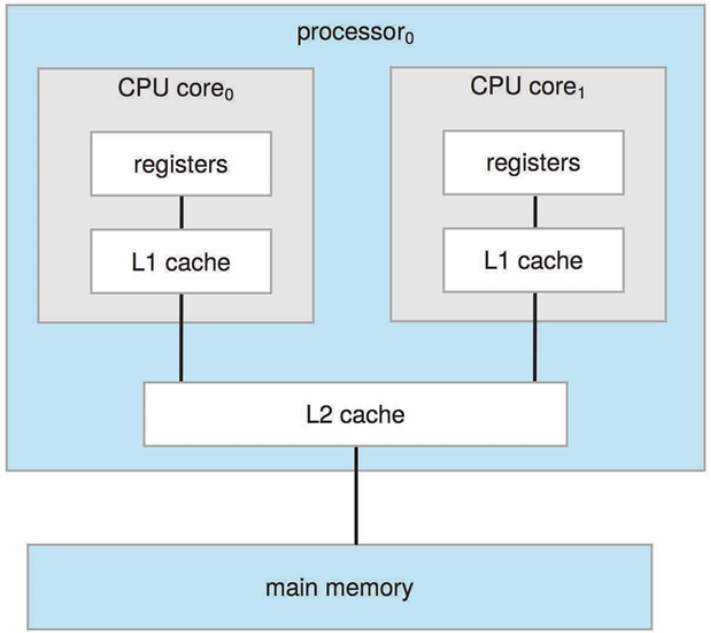


Figure 9: A dual-core design with two cores on the same chip.

Multiprocessor Systems

- Adding more CPUs to a multiprocessor system increases computing power, but scalability is limited. Once too many CPUs are added, contention for the system bus creates a bottleneck, degrading performance.
- An alternative approach is instead to provide each CPU (or group of CPUs) with its own local memory that is accessed via a small, fast local bus.
- The CPUs are connected by a shared system interconnect, so that all CPUs share one physical address space (non-uniform memory access or NUMA).
- A potential drawback with a NUMA system is increased latency when a CPU must access remote memory across the system interconnect.

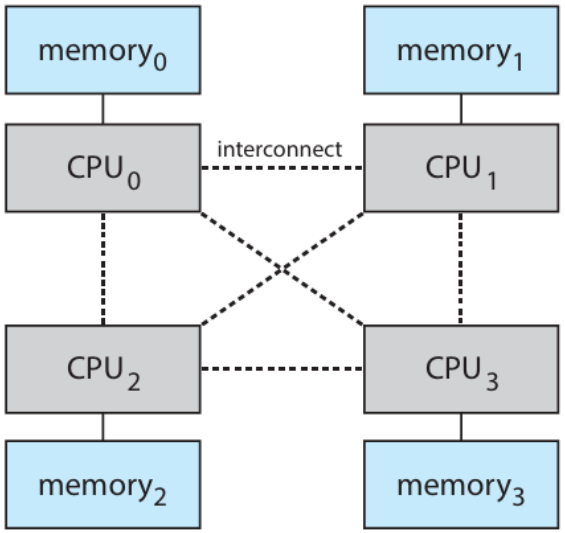


Figure 10: NUMA multiprocessing architecture.

Clustered Systems

- A clustered system is another type of multiprocessor setup that groups multiple CPUs together to work as a unified system.
- They are composed of two or more individual systems—or nodes—joined together; each node is typically a multicore system.
- Such systems are considered loosely coupled.
- Clustering is usually used to provide high-availability service—that is, service that will continue even if one or more systems in the cluster fail.
- We obtain high availability by adding a level of redundancy in the system. A layer of cluster software runs on the cluster nodes.
- Each node can monitor one or more of the others (over the network).
- The ability to continue providing service proportional to the level of surviving hardware is called graceful degradation.
- Some systems go beyond graceful degradation and are called fault-tolerant, because they can suffer a failure of any single component and still continue operation.

Clustered Systems

- Clustering can be structured asymmetrically or symmetrically.
- Asymmetric clustering, one machine is in hot-standby mode while the other is running the applications.
- The hot-standby host machine does nothing but monitor the active server.
- If that server fails, the hot-standby host becomes the active server.
- Symmetric clustering: multiple hosts run applications and monitor another.
- It is more efficient, as it uses all of the available hardware. However, it does require that more than one application be available to run.
- Parallelization is used to divide a program into separate components that run in parallel on individual cores in a computer or computers in a cluster.

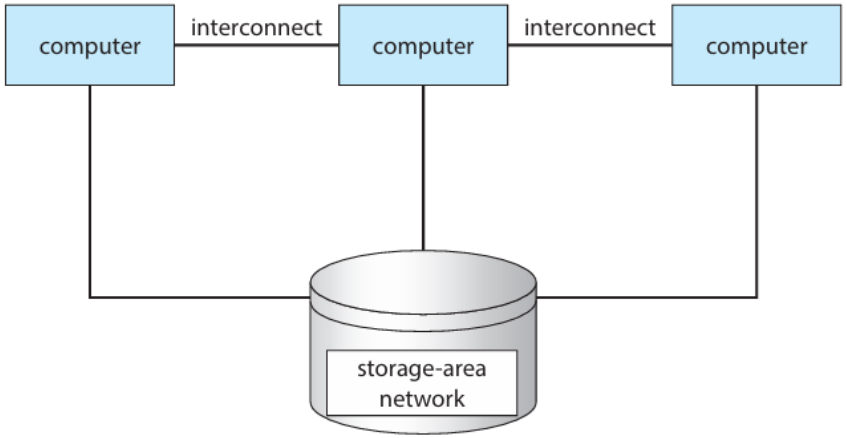


Figure 11: General structure of a clustered system.

Multiprogramming and Multitasking

- One of the most important aspects of operating systems is the ability to run multiple programs, as a single program cannot, in general, keep either the CPU or the I/O devices busy at all times.
- Furthermore, users typically want to run more than one program at a time as well.
- Multiprogramming increases CPU utilization, as well as keeping users satisfied, by organizing programs so that the CPU always has one to execute.
- In a multiprogrammed system, a program in execution is termed a process.
- The operating system keeps several processes in memory simultaneously. The operating system picks and begins to execute one of these processes.
- Eventually, the process may have to wait for some task, such as an I/O operation, to complete.
- In a non-multiprogrammed system, the CPU would sit idle. In a multiprogrammed system, the operating system simply switches to, and executes, another process until the first process finishes waiting and gets the CPU back

Multiprogramming and Multitasking

- Multitasking is a logical extension of multiprogramming.
- In multitasking systems, the CPU executes multiple processes by switching among them, but the switches occur frequently, providing the user with a fast response time.
- Eventually, the first process finishes waiting and gets the CPU back.
- Having several processes in memory at the same time requires some form of memory management.
- If several processes are ready to run at the same time, CPU scheduling is performed.

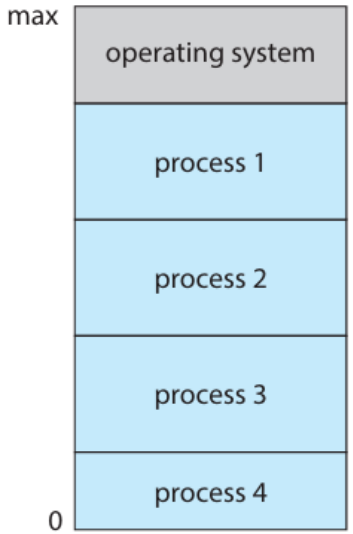


Figure 12: Memory layout for a multiprogramming system.

Thank you!

jaidoo@umat.edu.gh